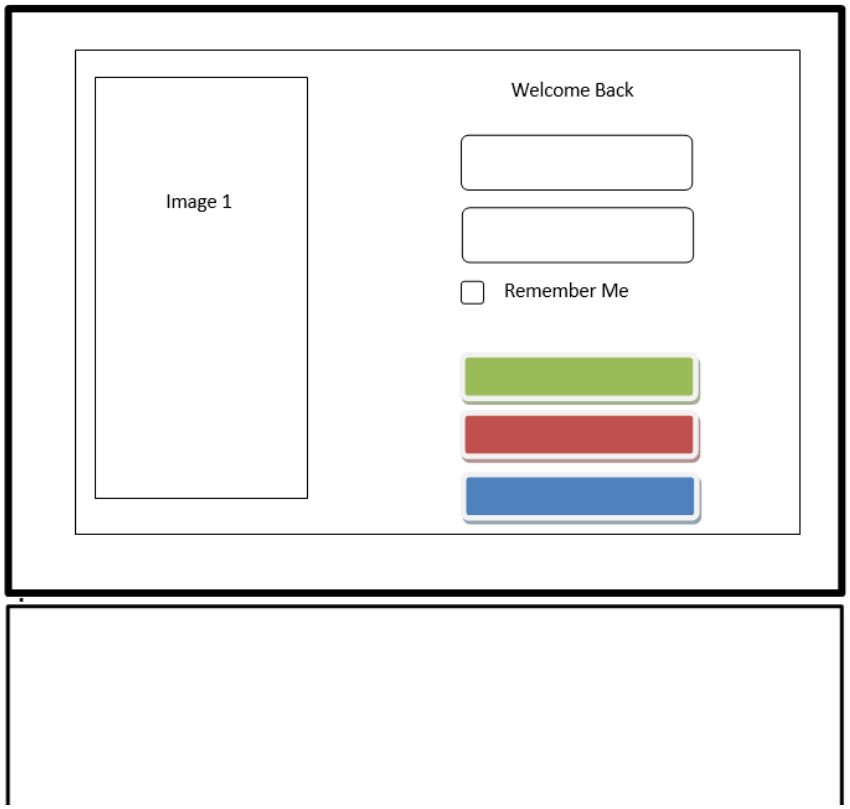
Task 1 Activity B the designs

Visual/interface designs

Login page



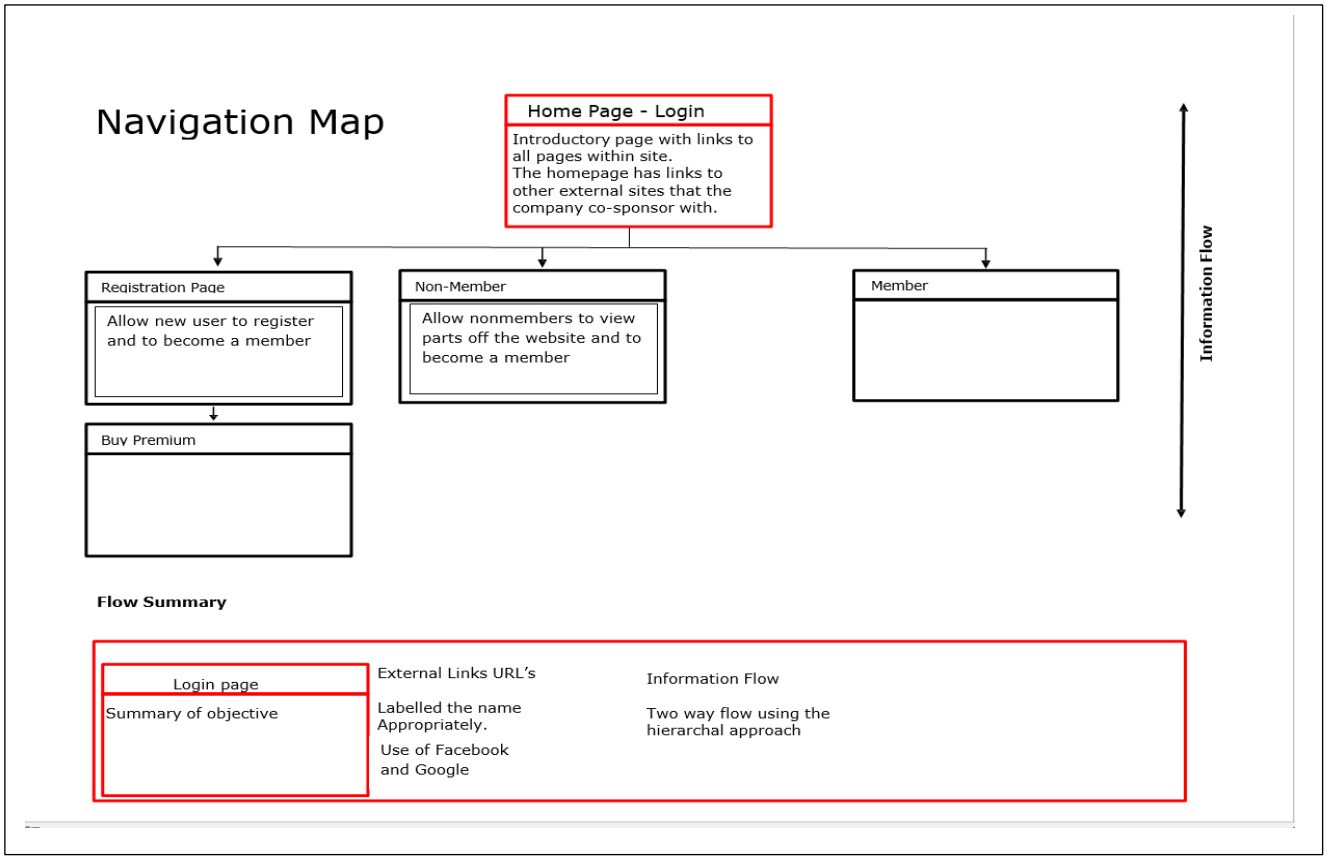
Page Specification

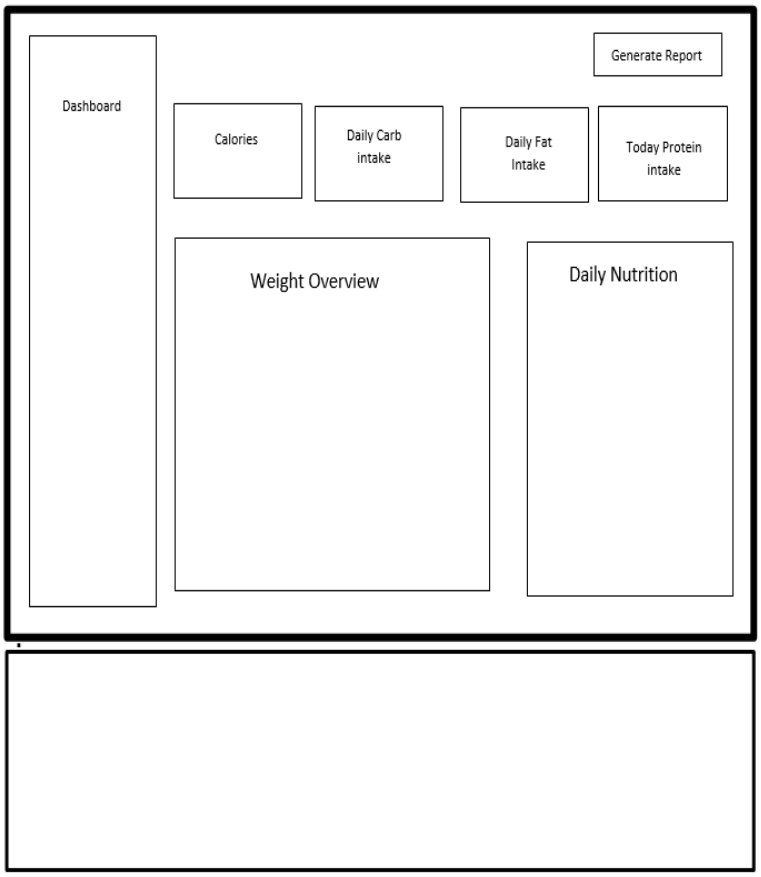
Interface

- Text Only- labels
- Global Hyperlinks – Google and Facebook for sign in
- Colour and typography: use company branding
  - Font type
  - Font colour
  - Background

Page Elements

- Use crumb trails
- Page dimension: considered
- Textual Areas
- Image Size
- Text boxes:
  - Username
  - Password
- Buttons:
  - Login
  - Google
  - Facebook





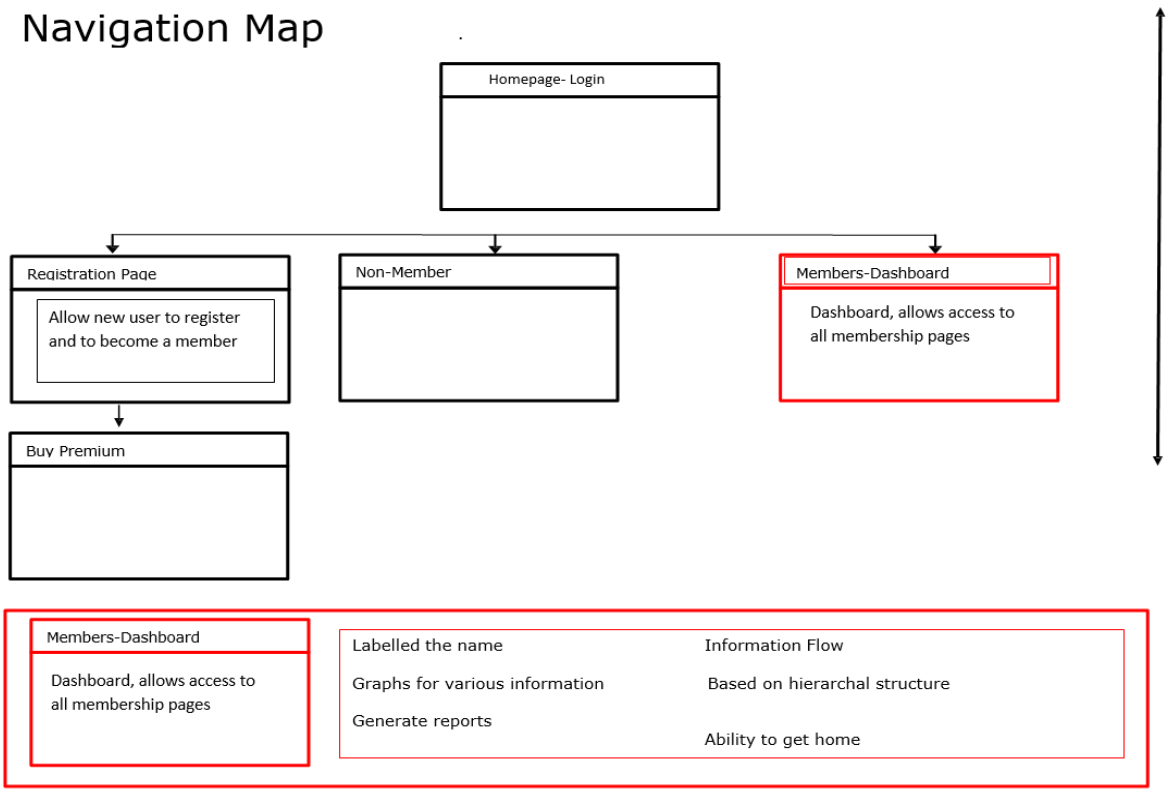
Page Specification

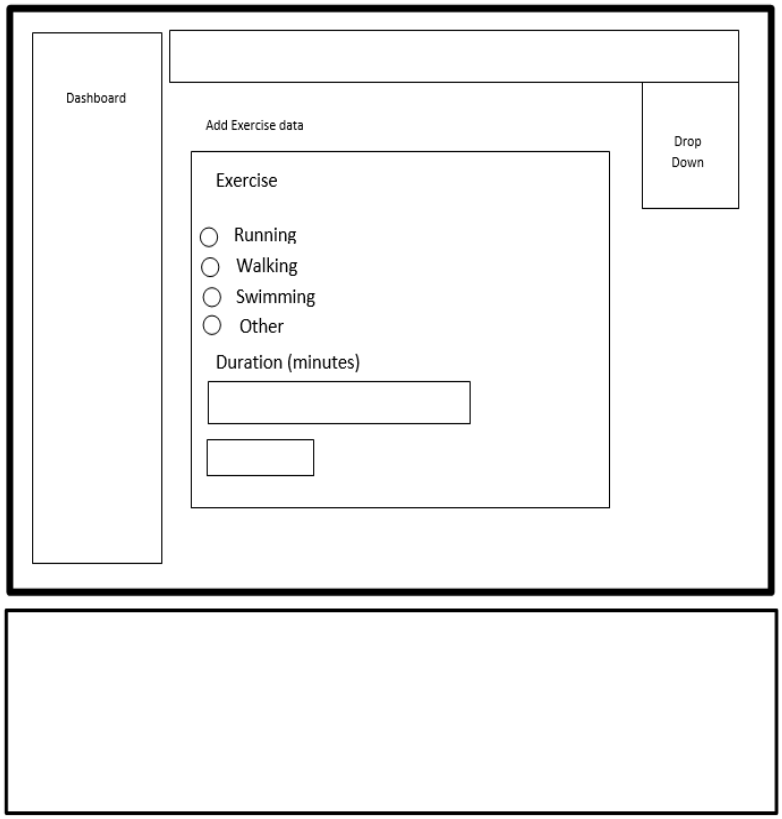
Interface

- Text Only- labels
- Global Hyperlinks –Dashboard
- Colour and typography: use company branding
  - Font type
  - Font colour
  - Background

Page Elements

- Use crumb trails
- Page dimension: considered
- Textual Areas
- Image Size: considered
- Bar Graphs
  - Calories
  - Carb
  - Fat
  - Protein
- Weight Overview
  - Y = Weight
  - X = Months





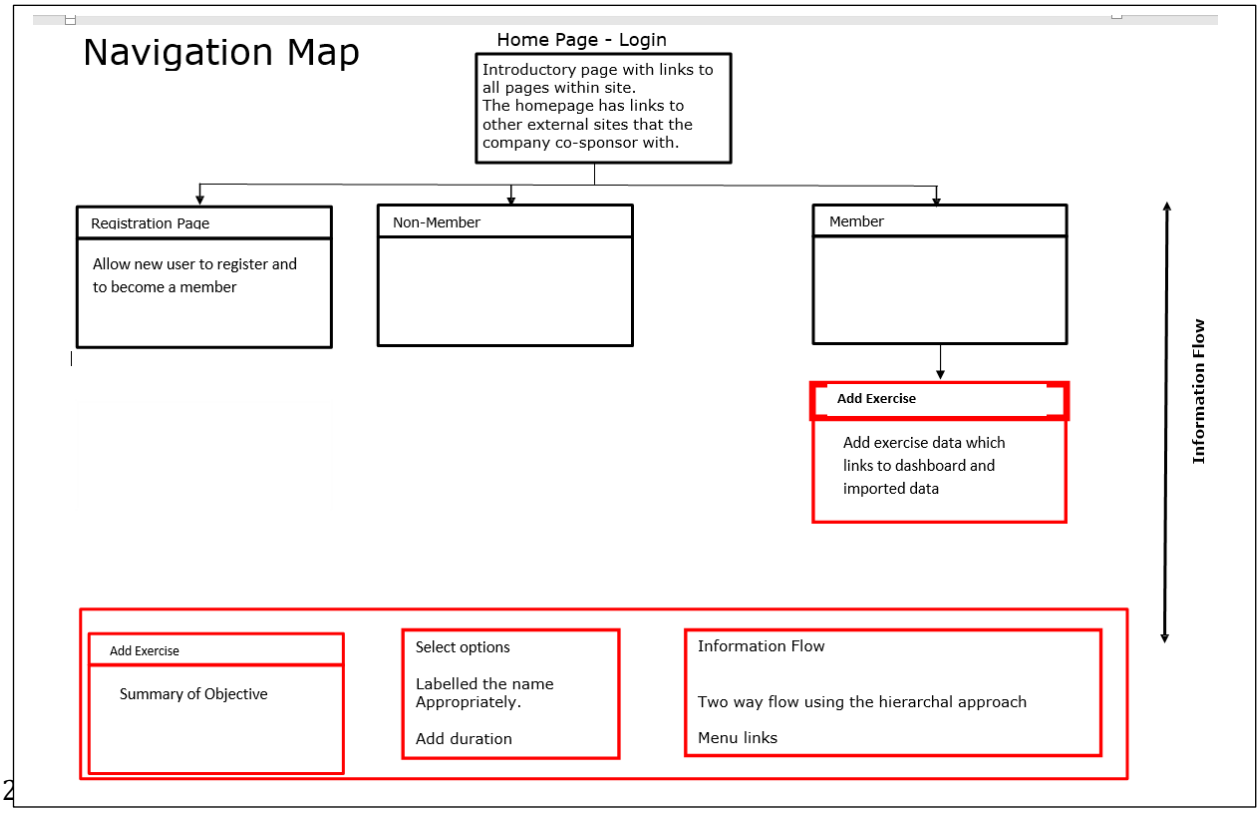
Page Specification

Interface

- Text Only- labels
- Global Hyperlinks – Tracker diet
- Colour and typography: use company branding
  - Font type: TBC
  - Font colour: Grey
  - Background: White & Green

Page Elements

- Use crumb trails
- Page dimension: considered
- Textual Areas: None
- Drop down:
  - Profile
  - Settings
  - Activity Log
  - Logout
- Add Exercise
  - Labels: running, walking Swimming, other
  - Radio Button
  - Text Box
  - Button - submit



Dashboard

Generate Report

Food Diary

Date	Food	Weight	Calories	Protein	Carbs	Fat

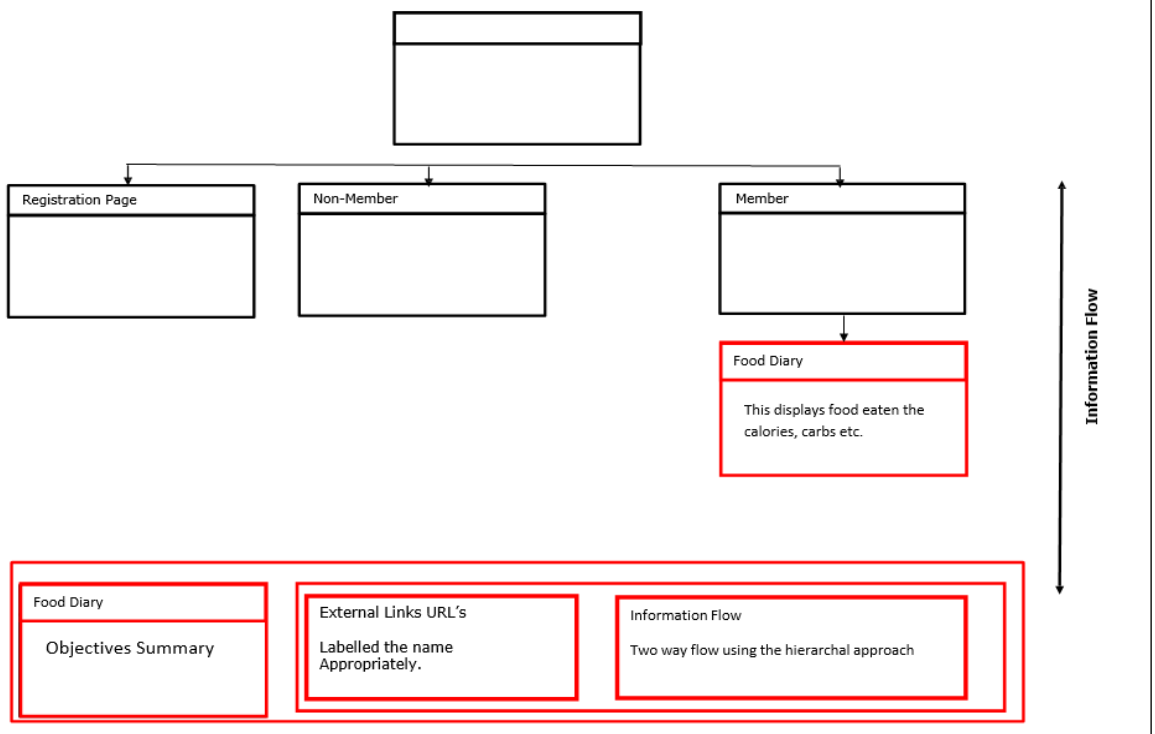
Page Specification

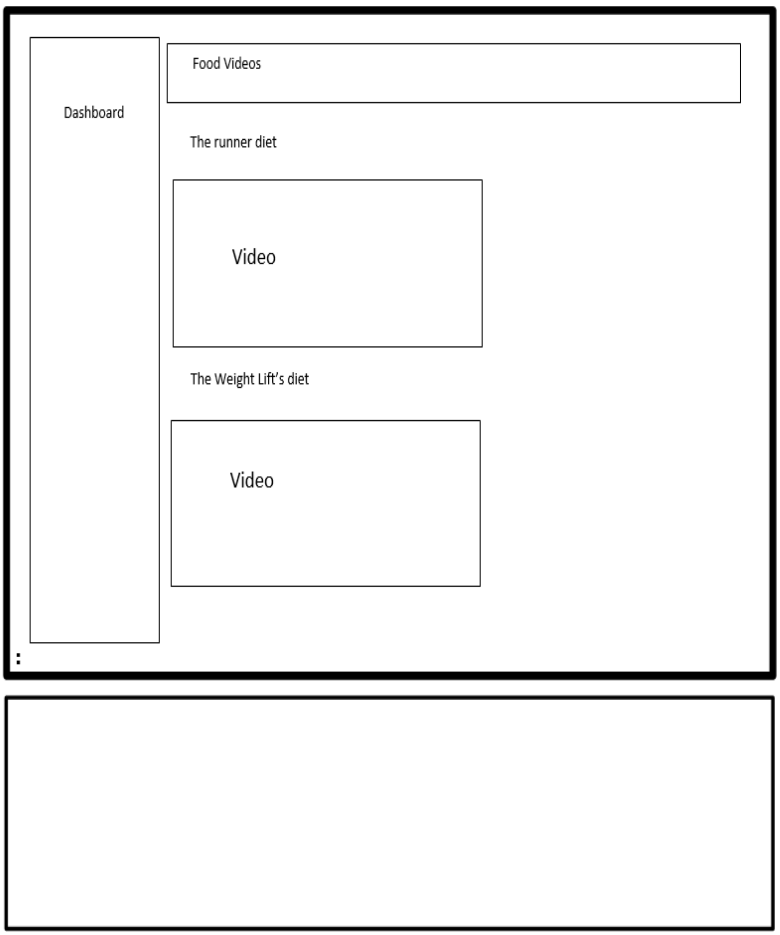
Interface

- Text Only- labels
- Global Hyperlinks – Dashboard
- Colour and typography: use company branding
  - Font type: TBC
  - Font colour: Grey
  - Background: White & Green

Page Elements

- Use crumb trails
- Page dimension: considered
- Textual Areas: None
- Table: history
  - Date
  - Food intake
  - Weight(g)
  - Calories
  - Protein(g)
  - Carbs(g)
  - Fat(g)





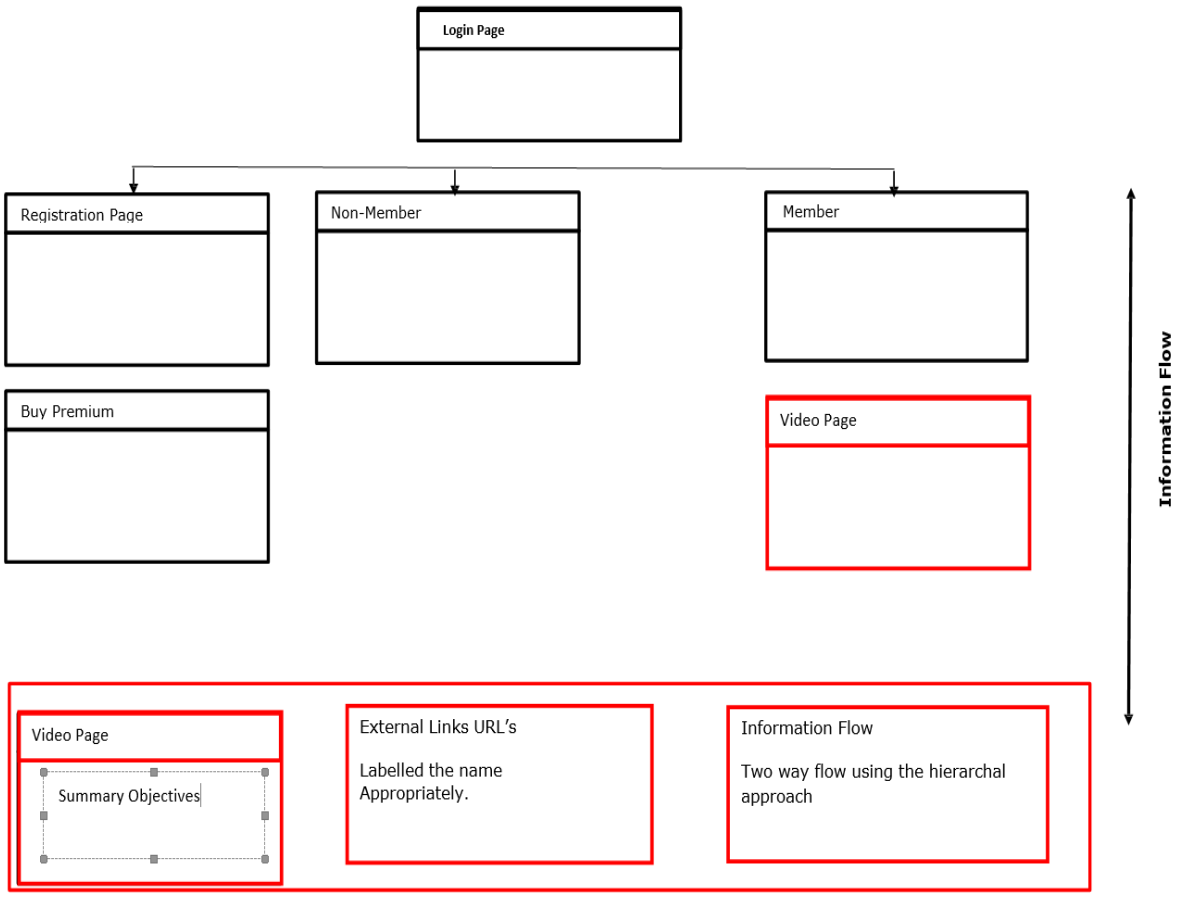
Page Specification

Interface

- Text Only- labels
- Global Hyperlinks – Dashboard
- Colour and typography: use company branding
  - Font type: TBC
  - Font colour: Grey
  - Background: White & Green

Page Elements

- Use crumb trails
- Page dimension: considered
- Textual Areas: None
- Multimedia:
  - Video 1
  - Video 2



Dashboard

Fitness Diary

History

Date	Exercise	Calories	Duration

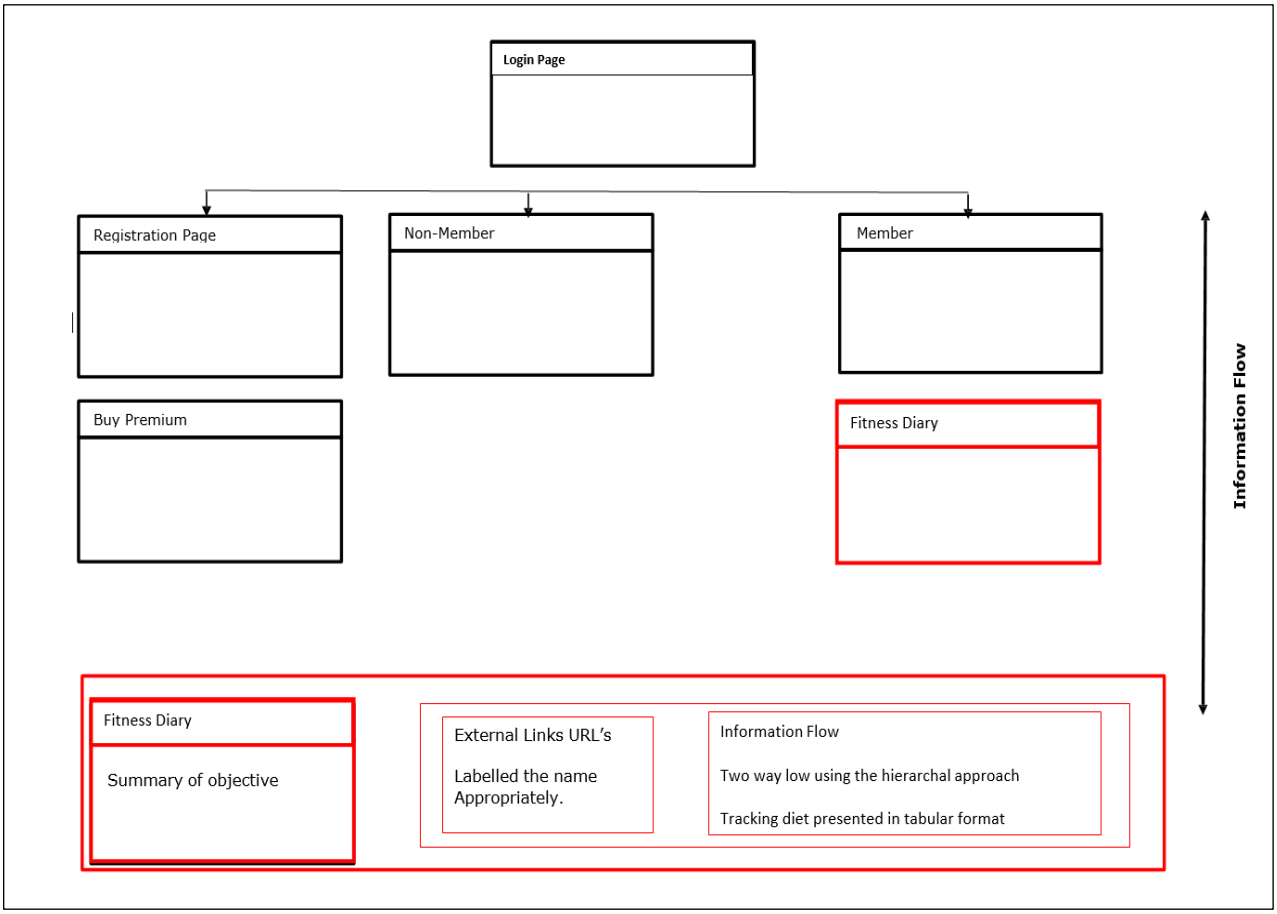
Page Specification

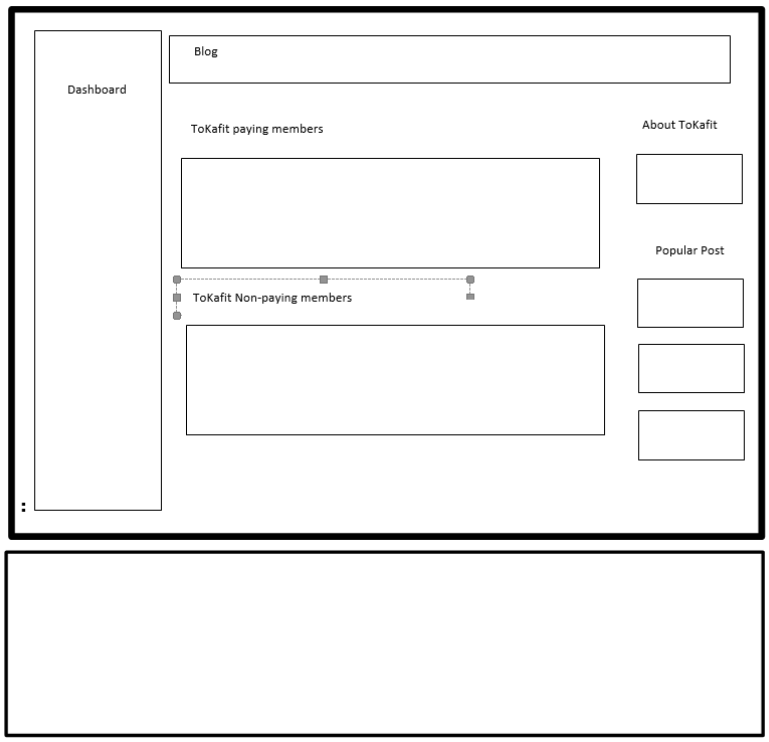
Interface

- Text Only- labels
- Global Hyperlinks – Dashboard
- Colour and typography: use company branding:
  - Font type: TBC
  - Font colour: Grey
  - Background: White & Green

Page Elements

- Use crumb trails
- Page dimension: considered
- Textual Areas: None
- Tabular Data
  - Date
  - Exercise
  - Calories
  - Duration





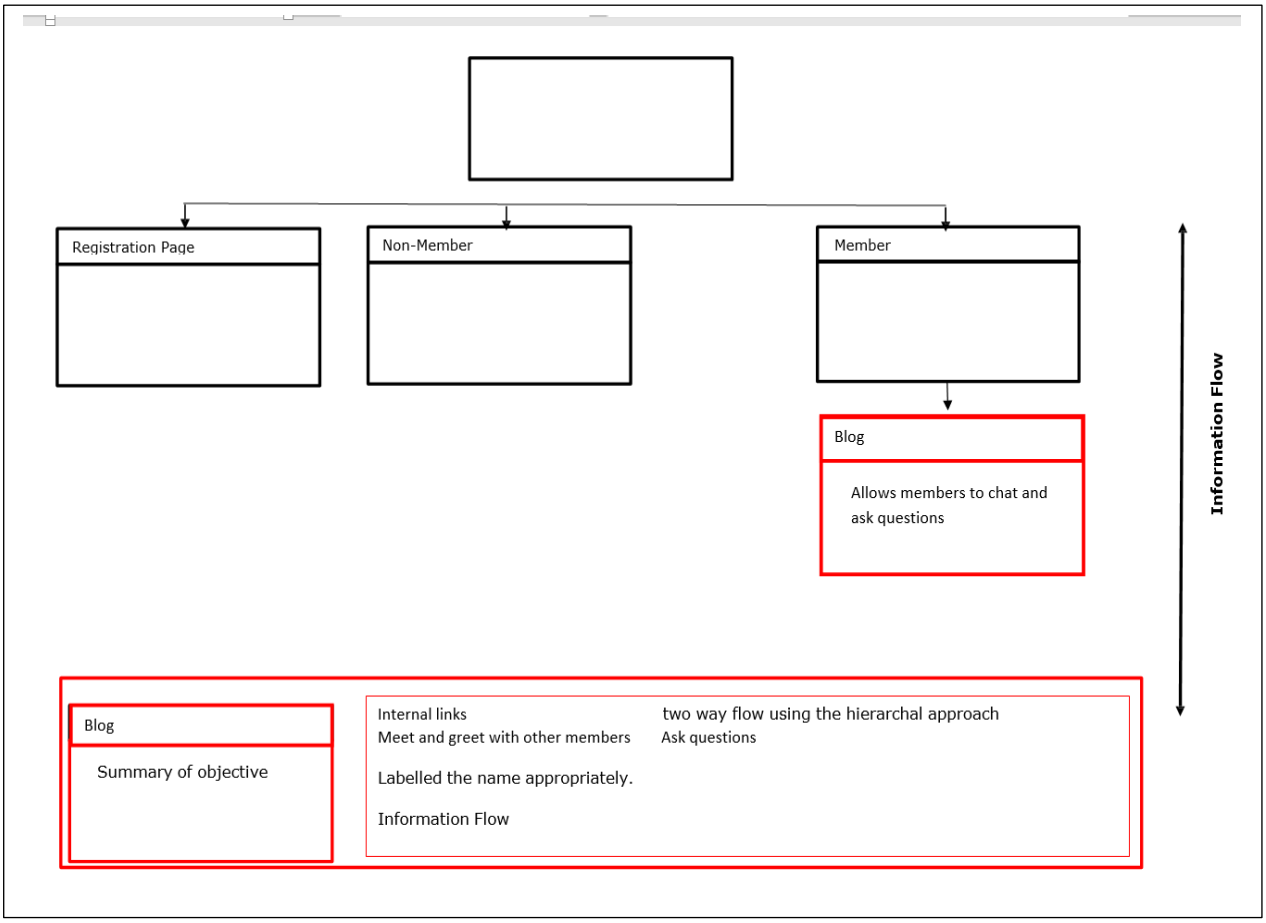
Page Specification

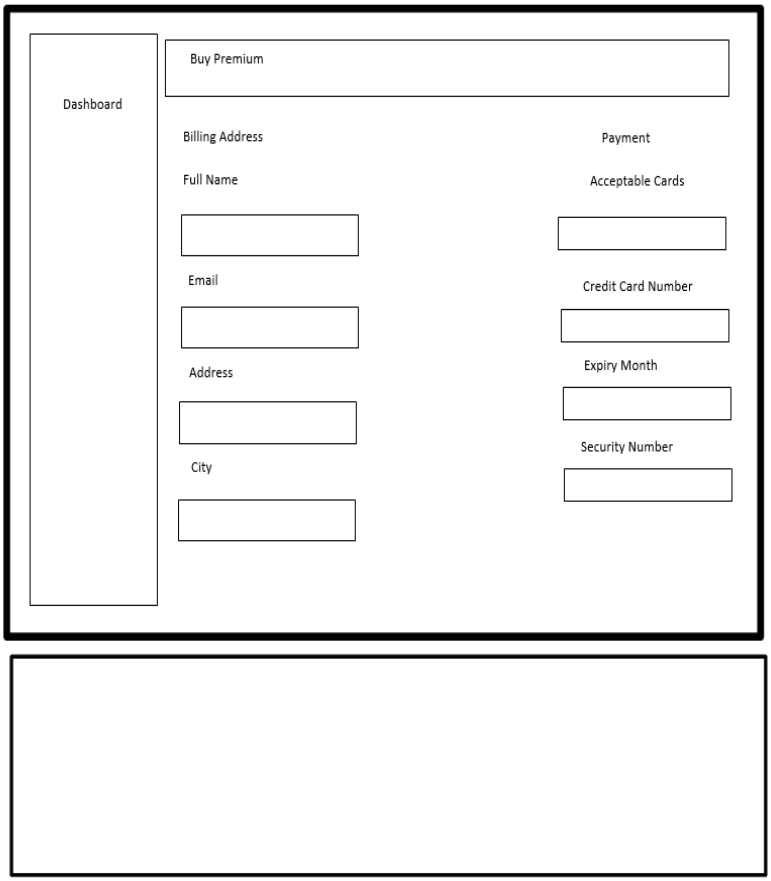
Interface

- Text Only- labels
- Global Hyperlinks – Dashboard
- Colour and typography: use company branding
  - Font type: TBC
  - Font colour: Grey
  - Background: White & Green

Page Elements

- Use crumb trails
- Page dimension: considered
- Textual Areas: None
- Form:
  - Text area 1
  - Text area 2
  - Popular Post





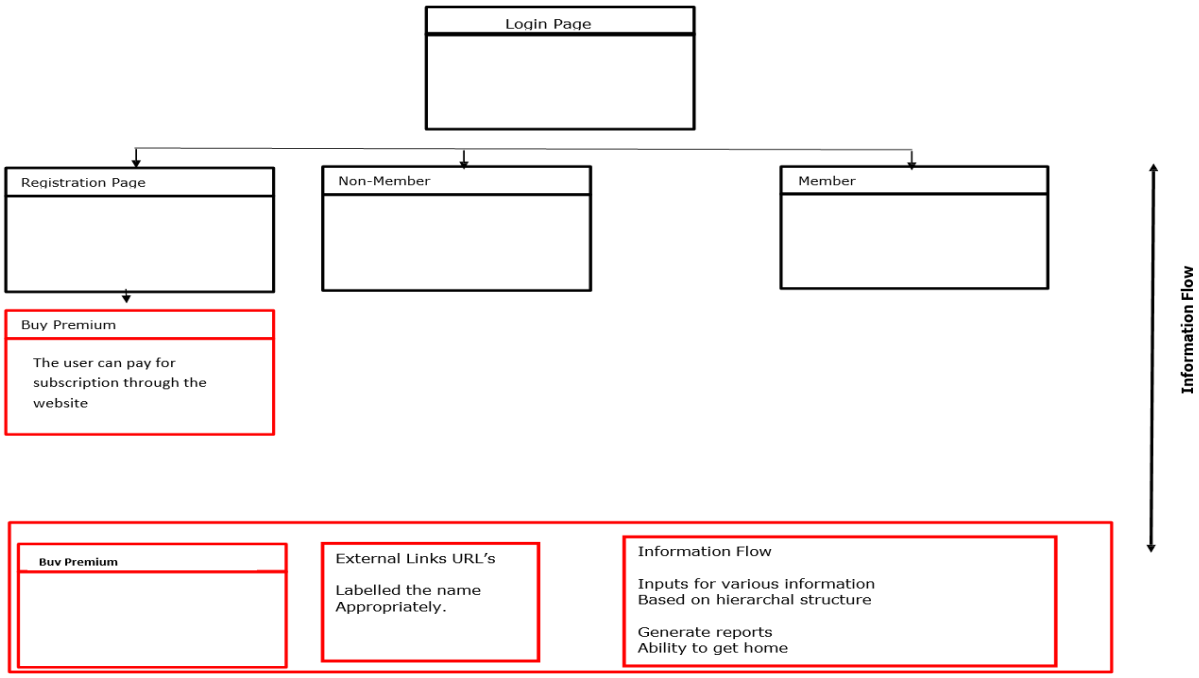
Page Specification

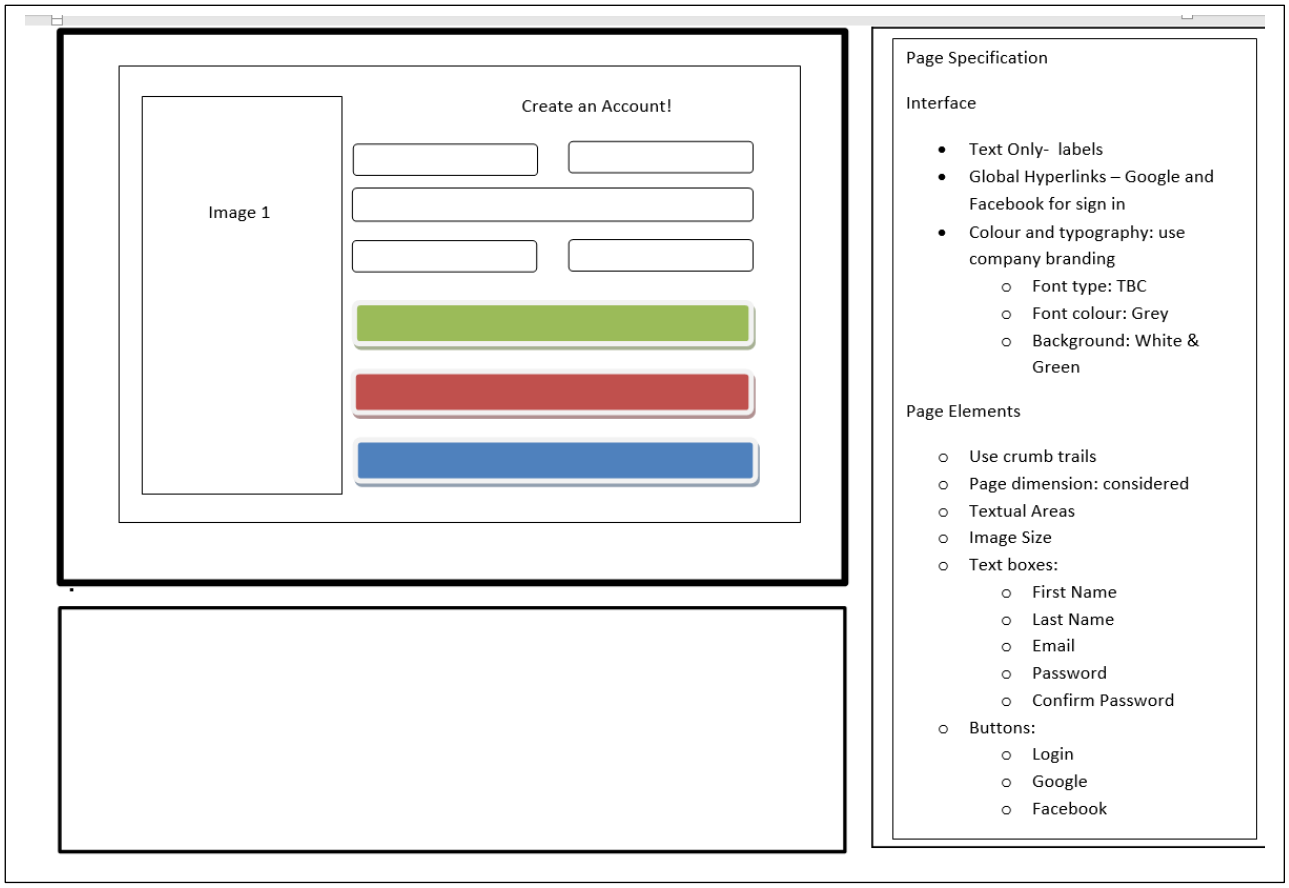
Interface

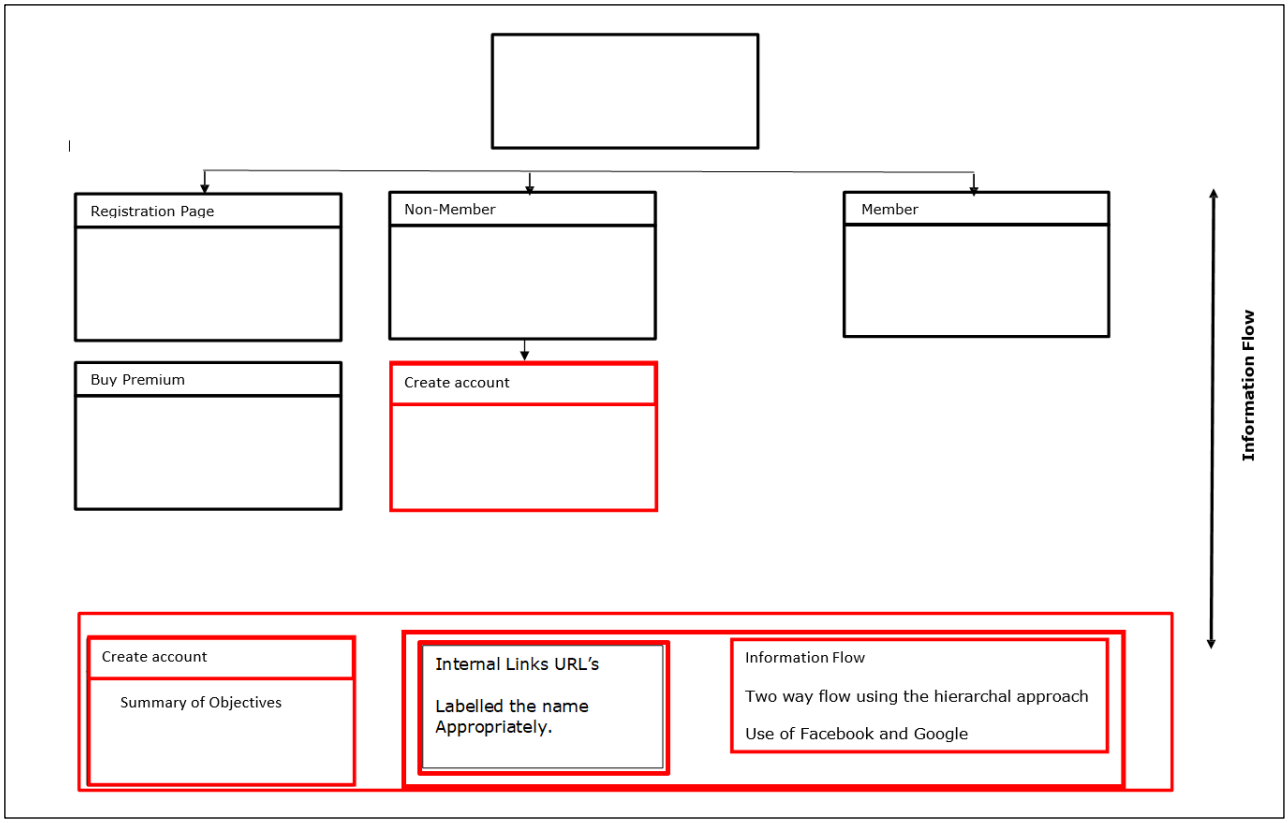
- Text Only- labels
- Global Hyperlinks – Dashboard
- Colour and typography: use company branding
  - Font type: TBC
  - Font colour: Grey
  - Background: White & Green

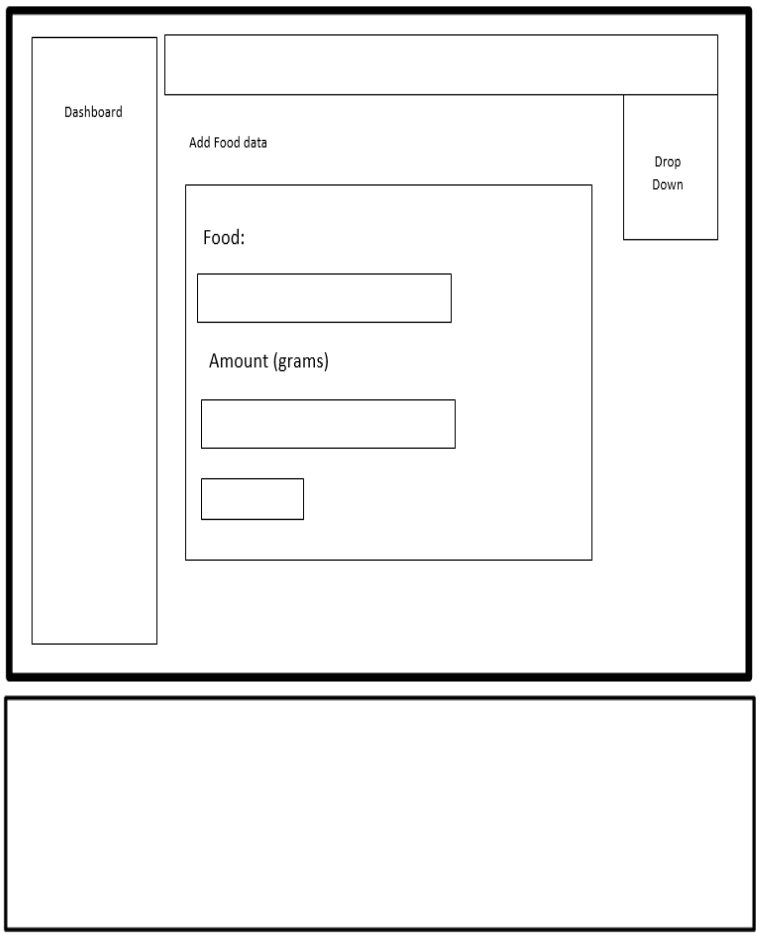
Page Elements

- Use crumb trails
- Page dimension: considered
- Textual Areas: None
- Form:
  - Text area 1
  - Text area 2
  - Popular Post




Create an Account!
Image 1
Page Specification
Interface
Text Only- labels
Global Hyperlinks – Google and Facebook for sign in
Colour and typography: use company branding
Font type: TBC
Font colour: Grey
Background: White & Green
Page Elements
Use crumb trails
Page dimension: considered
Textual Areas
Image Size
Text boxes:
First Name
Last Name
Email
Password
Confirm Password
Buttons:
Login
Google
Facebook


Registration Page
Non-Member
Member
Buy Premium
Create account
Information Flow
Create account
Summary of Objectives
Internal Links URL's
Labelled the name Appropriately.
Information Flow
Two way flow using the hierarchal approach
Use of Facebook and Google



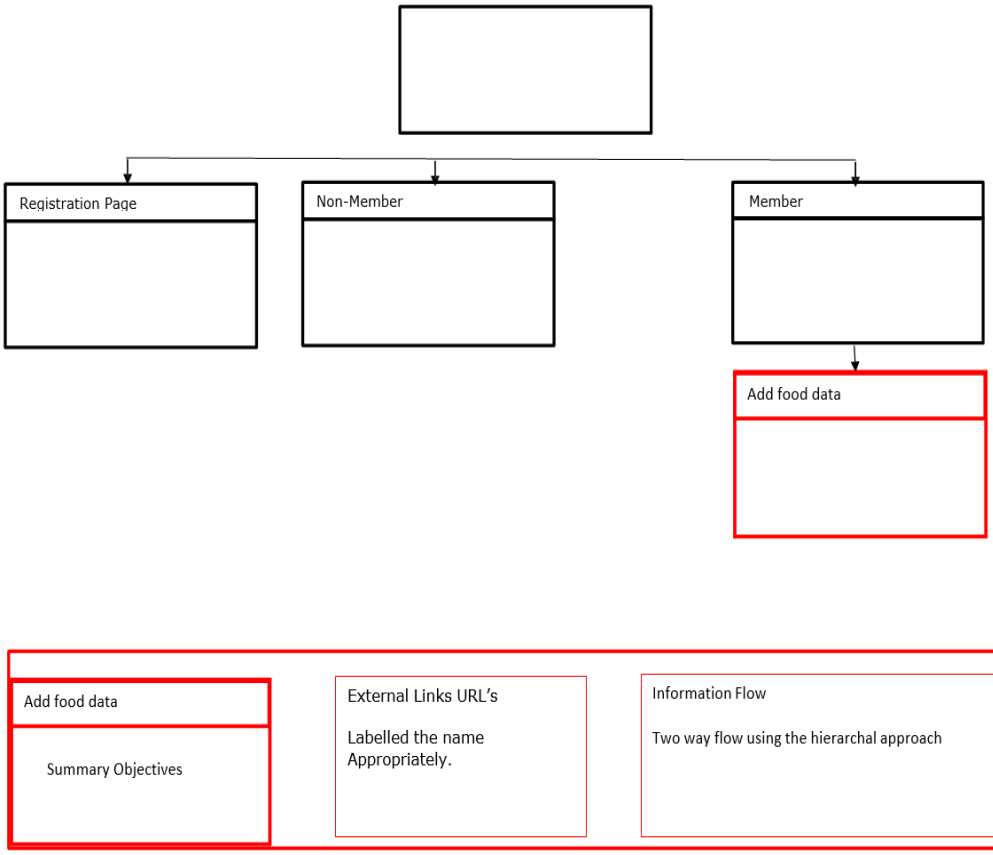
Page Specification

Interface

- Text Only- labels
- Global Hyperlinks – Tracker diet
- Colour and typography: use company branding
  - Font type: TBC
  - Font colour: Grey
  - Background: White & Green

Page Elements

- Use crumb trails
- Page dimension: considered
- Textual Areas: None
- Drop down:
  - Profile
  - Settings
  - Activity Log
  - Logout
- Add Exercise
  - Labels: food, amount
  - Text boxes



Front End Needs

- Different icons that are easy for the user to use
- Attractive
- Clear link to the back end (not for the user but for management to ensure that they can receive data that users input)

Back End Needs

- Data storage
- Reading/writing to and from this
- Stored in an efficient way
- Security (hashing, encryption)
- Link to the front end

Security and data analysis

System security will be an important issue for management, just as it will be for the customers. Overall, some of management's needs do overlap with other stakeholders although management will be the ones who have to face the consequences of any breaches of the system. This means that it's their responsibility to ensure that the system they're providing to their customers is safe to use. Organising the data will benefit management when it comes to producing reports and carrying out queries. This is important because during periods where there are lots of customers, management will be able to quickly carry out queries and find out the information they need. The data analysis section will make up parts of the report as well as a different section of the solution. This part is supposed to aid decision making about the whole restring process, because it will impact which workout they choose to use. From the management's perspective the reason behind this data analysis is to help them reduce their costs and to improve the overall efficiency of the system.

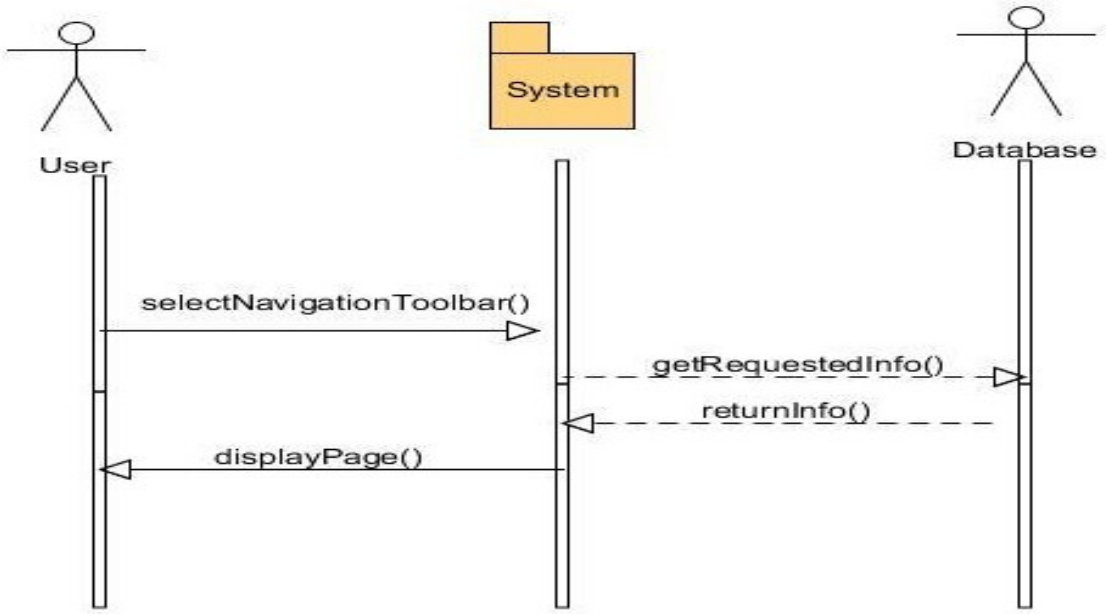
Algorithms

```
if(first time entry)
{
  Load registration form
}
else
{
  Res_userID = check whether input userID is unique

  res_pwd    = check whether input password1 and input password2 are the same

  res_email = check whether input email is of correct format
```

```
res_profile = check whether input profile is not empty

res_name     = check whether input name field is not empty

if ( all check results == true)
{
        SQL insert queries

         connect to database

        pass queries to database

                if (any query fails)
                        display error msg

        else display registration complete msg
        endif
{

        if (res_userID == false)
                display message reselect unique user id
        endif

        if (res_pwd== false)
                display message retype password
        endif

        if (res_name== false)
                display message retype name
        endif)

        if (res_email  == false)
                display message retype a valid email
        endif
```

```
        if (res_profile  == false)
                display profile is empty msg
        endif
  }
        endif
}
endif

end Registrations module
```

Test case	Path	Variable Value	Expected result
1	1	first time entry == 1	- load registration form - exit
2	2	first time entry == 0 && all check results == 1 && any queries == fail	- execute node 4 - generate error msg - exit
3	3	first time entry == 0 && results = true && query == true	- execute node 4 - display registration complete message - exit
4	4	first time entry = 0 && all check results = 0	- load registration form
5	5	userID == false	- display error msg
6	6	User ID == true	- go on
7	7	Password == false	- display error msg
8	8	Password == true	- go on
9	9	name == false	- display error msg
10	10	name == true	- go on
13	13	Email == false	- display error msg
14	14	Email == true	- go on



Login Screen:

START

clear username textbox

clear password textbox

INPUT Username

INPUT Password

count = 3

WHILE count is not equal to 0

print 'the number of attempts left is' count

IF username = username in vault file

AND

IF password = password in vault file

open index

else print 'username ort password is invalid'

count = count -1

REPEAT until no more counts available

message 'please contact administrator'

SQL statements:

SELECT * FROM customer WHERE Username = “username” AND Password = “password”

Justification: The start clears the username and password to allow the user to login to the website. As it progresses, the while loop is used to prevent an automated robot from logging into random accounts. If the password and username are correct they are allowed to the main page, if not then the session will time out.

Validating Email

```
Function ValidateEmail;

 If local length <1 or >64

        Output Invalid email

If domain length <1 or > 255

        Output invalid email

If character (0)  equal to”.” Or character (-1)  equal to ’.’

        Output invalid email

If email contains ‘..’

        Output invalid email

If email contain characters other than (A-Z,a-z,0-9) Output invalid email

Else email is valid

end function
```

The reason to validate the email is to ensure that the user inputs the correct format, so that the details are correctly put in. A key will be sent to the email address to validate that this is their email, so if they put the wrong email address in they can’t access it and therefore won’t be able to get the

Validation of the password is to ensure the user creates a password. To make sure the password is sensible and strong is by making sure the password is over 6 characters long. There must be number in the validation as well as a capital and lowercase letter, increasing the strength of the password. The validation also compares the password to the confirmed password to make sure they are the same. This makes sure the user has entered the correct password, therefore if the user has made an error typing in the password it won't be loaded into the database incorrectly.

Validating Password

Function ValidatePassword

If length greater or equal to 6

Output 'password must be over 6 characters'

If password not equal to (0-9)

Output 'password must contain a number'

If password not equal to (A-Z)

Output 'password must contain a capital letter

If password not equal to (a-z)

Output 'password must contain a lowercase letter

If password is not equal to Confirm password

Output 'the password entered doesn't match'

Else password is valid

End function

Hashing Password

Alphabet = ['ABCDEFGHIJKLMNOPQRSTUVWXYZ'] //List of characters to replace password with.

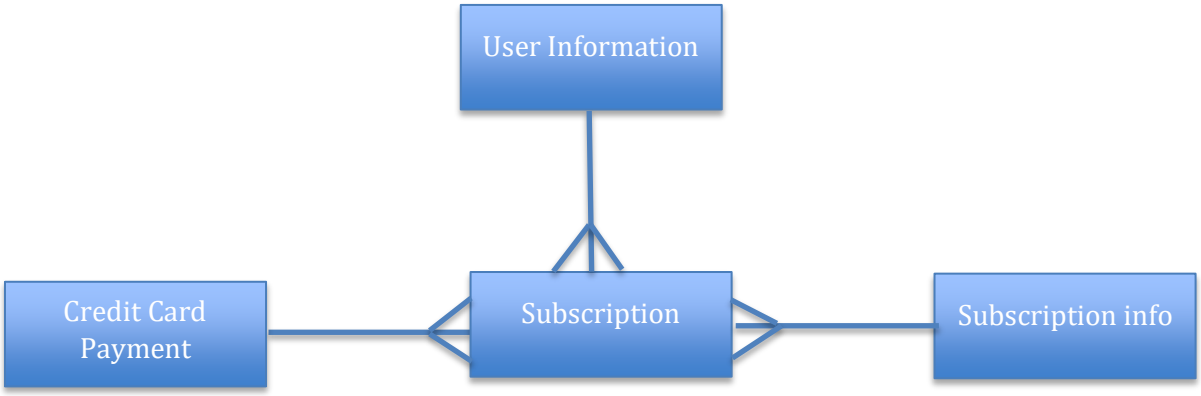
Numbers = [0123456789] //List of numbers to replace password with.

```
CREATE PROCEDURE hashPassword

        WITH CONNECT ('Accounts.db') AS Database //

        CreateAccount () //Calls this function because this is where the password is initially
        created.

system

Submit.onclick() //When the create account form (password) is submitted to the

                FOR I in range 0 to len (Password) //Checks length of password, which is
                retrieved        from the create account function.

 N = 0
WHILE N != len(Password) //Keeps replacing characters until the length of the password is
met.


HashedPassword = Password.ammend (randomchar[alphabet, numbers]) //Replaces
characters with either letters or numbers.

hashed.

N = N+1 //Increments n, repeats until length of password has been

                        ELSE BREAK //Ends the loop once fully

                        hashed. INSERT INTO Accounts

                        Values (FirstName, Surname, Mobile, Email, HashedPassword);
//Stores these values inside the table.
 END Procedure

 CREATE Procedure loginWithHashedPassword()
 Login () //Allows the user to login with their plaintext password.
 HashPassword (Password) //Hashes the input in order to see if they match.
```

Data Requirements

Variable name	Function	Data type	Reason
$isValid	To determine if email entered by user is valid	Boolean	As email entered is either valid(true) or not valid(false)
$atIndex	As a form of validation to make sure the email entered has	String	To locate position of @ sign
$domain	To do a length check on the domain of the email	String	To assign the whole email to a variable
$local	To assign the part of the email before the @ sign to a variable	string	To help validate the first part of the email
$localLen	Counts the number of characters in the local part of the email	string	To validate the length of the $localLen
$domainLen	Counts the number of characters in the domain	string	To validate the length of the domain
$password	Assigning the user's	string	To allow for validation

	input for a password to a variable		to the password.
$cpassword	Assigning the user's input to confirm password to a variable	String	To allow for validation of the password
$passwordErr	To output a message when validation of password isn't met	string	If there is an error then it outputs a message
$confirmpErr	To output a message when confirmed password is check against password	String	To output an error when confirmed password doesn't match password
$nameErr	To check if the field "name" is filled	string	To make sure there is a present value
$name	Assigns the input to the field "name"	string	To assign a value to the variable name
$surnameErr	To check if the field "surname" is filled	string	To make sure there is a present value
$surname	Assigns the input to the field "surname"	string	To assign a value to the variable name
$data		string	



Credit Card Accounts	
Field	**Data Type**
Account No.	int (10)
Username	varchar
Password	varchar
CustomerID	int (6)

UserInfo	
Field	**Data Type**
UsernameID	int (6)
Surname	varchar (25)
First Name	varchar (20)
Telephone	Varchar(13)
Profileimg	

Subscription	
Field	**Data Type**
Username	Int(6)
SubID	Int(15)
PayID	Int(15)
SubStart	Date/time
SubEnd	Date/time
AutoRenew	Boolean

Approach to testing

To test my solution, I will be using white box testing (where I test the internal structure of the website) and black box testing (where I only test the inputs and outputs).

Test strategy	White box testing
Purpose	This test strategy is a form of structural testing. The testing is dependent on the code logic and assesses the program structure rather than the program function. The program will be analysed, and tests will be devised to test every possible pathway at least once. In simple terms this tests that each part of the code itself functions as intended.
Who performs the test	Developer
Test data set	All possible inputs. Outputs to each input will be recorded.
Test criteria	The output should be as intended by the programmer (me) for each input.
When to test	White box testing will be included in nearly all testing phases.
Estimated time required	Individual inputs can be tested in less than a minute. Depending on stage of development, full white box testing and documentation of results for whole program could be more than one hour.
Test outcome	Outcome data will be compared to intended outcome for each coding path, the results will be used to guide any alterations and improvements to each code that are required to ensure that each code pathway leads to the required outcome.

```
12  */
13 (function( factory ){
14     if ( typeof define === 'function' && define.amd ) {
15         // AMD
16         define( ['jquery', 'datatables.net'], function ( $ ) {
17             return factory( $, window, document );
18         } );
19     }
20     else if ( typeof exports === 'object' ) {
21         // CommonJS
22         module.exports = function (root, $) {
23             if ( ! root ) {
24                 root = window;
25             }
26 
27             if ( ! $ || ! $.fn.dataTable ) {
28                 // Require DataTables, which attaches to jQuery, including
29                 // jQuery if needed and have a $ property so we can access the
30                 // jQuery object that is used
31                 $ = require('datatables.net')(root, $).$;
32             }
33 
34             return factory( $, root, root.document );
35         };
36     }
37     else {
38         // Browser
39         factory( jQuery, window, document );
40     }
41 }(function( $, window, document, undefined ) {
42  'use strict';
43  var DataTable = $.fn.dataTable;
44 
45 
```